

TP - 01. Authentification OTP - OTP via application mobile

Propriétés (6)	Valeurs
Bloc	Bloc3
version	0.4
Compétences du référentiel	B3.1. Protection des données à caractère personnel , B3.1.4 - Sensibilisation des utilisateurs à la protection des données à caractère personnel , B3.2.1 - Protection de l'identité numérique d'une organisation , B3.3.1 - Information des utilisateurs sur les risques associés à l'utilisation d'une ressource numérique et promotion des bons usages à adopter , B3.3.3 - Gestion des accès et des privilèges appropriés , B3.3.4 - Vérification de l'efficacité de la protection
Sources	https://sebsauvage.net/wiki/doku.php?id=totp , https://freeotp.github.io ,  Memo - 01. Authentification Multi Facteurs (MFA - 2FA) , https://github.com/LiliwoL/PHP-Authentification-OTP
Calendrier	Semestre4
tags	B3, authentification, autoévaluation



Introduction

L'authentification d'un utilisateur se fait habituellement avec **login** et **mot de passe**.

On peut améliorer la sécurité en ajoutant des **facteurs**.

On en compte habituellement 3:

- Ce que je **sais** (*Mon mot de passe*)
- Ce que je **possède** (*un objet, une clé, un token...*)
- Ce que je **suis** (*biométrie*)

Si la **biométrie** nécessite souvent du matériel spécialisé (lecteur d'empreinte digitale, iris, rétinienne...), on peut facilement se créer l'équivalent d'un token d'authentification avec son smartphone (ce que je possède).

Code à usage unique (OTP)

Il existe des applications respectant un protocole standardisé pour cela: **OTP** (*One Time Password*).

L'intérêt de l'OTP est que vous n'avez **pas besoin de connexion** entre votre client OTP (*votre smartphone*) et le serveur.

Ils peuvent **générer** et **contrôler** la validité des OTP de manière déconnectée.

Ce système peut donc fonctionner même si vous n'avez pas de réseau GSM à portée.

Ce protocole permet de créer un code qui ne sera **utilisable qu'une seule fois**.

Pour ce TP, nous allons juste **ajouter un champ de saisie dans le formulaire de login**.

Côté serveur

Côté serveur, nous allons utiliser la librairie **otphp** fournie par **spomky-labs**.

| <https://packagist.org/packages/spomky-labs/otphp>

Côté client

Côté client, sur votre téléphone ou tablette, les applications suivantes sont utilisables:

- **FreeOTP** (*disponible sous Android et iOS*)
- **Google Authenticator**
- **Microsoft Authenticator**
- Aegis
- Proton Authenticator

Pour la suite, du TP, nous utiliserons **FreeOTP**, mais vous pouvez essayer avec les applications propriétaires, ça fonctionnera!

Quel type d'OTP ?

Il existe deux type d'OTP: **HOTP** et **TOTP**.

HOTP est basé sur un compteur

A chaque utilisation, le **compteur est incrémenté** côté client et côté serveur.

L'inconvénient est que s'il y a un raté, il faut resynchroniser les compteurs.

TOTP est basé sur la date et l'heure

A partir du moment où le client et le serveur sont synchronisés, on peut générer autant d'OTP qu'on le souhaite.

Ces OTP ont une **durée de validité limitée** (qu'on peut paramétrer).

Nous allons implémenter **TOTP**.

Premier test

 Mise en place de la double authentification par application TOTP

Face aux risques accrus de cyber malveillance, de nouvelles mesures de sécurité et de vigilance sont prises par Parcoursup pour renforcer la sécurité numérique et protéger les données individuelles des candidats. **A compter de début Avril 2024, un protocole de double authentification est mis en place à chaque connexion au site de Gestion de Parcoursup.**

Si vous vous connectez depuis l'étranger nous vous invitons à utiliser une application TOTP qui permet de synchroniser les fuseaux horaires (ex : Google Authenticator) afin d'éviter tout problème de désynchronisation.

Étape 1 - Choisissez et installez sur votre smartphone une application TOTP



L'application la plus connue de TOTP est Google Authenticator, mais bien d'autres solutions gratuites existent sous Android ou iOS (Free OTP, AndOTP, Microsoft Authenticator, OTP Auth. ...).

Un code TOTP (Time based One Time Password) est un mot de passe à usage unique (OTP) généré depuis une application sur votre smartphone.

Étape 2 - Ouvrez l'application TOTP avec votre smartphone et scannez ce QR Code





Vous avez des questions : consultez la [FAQ](#).

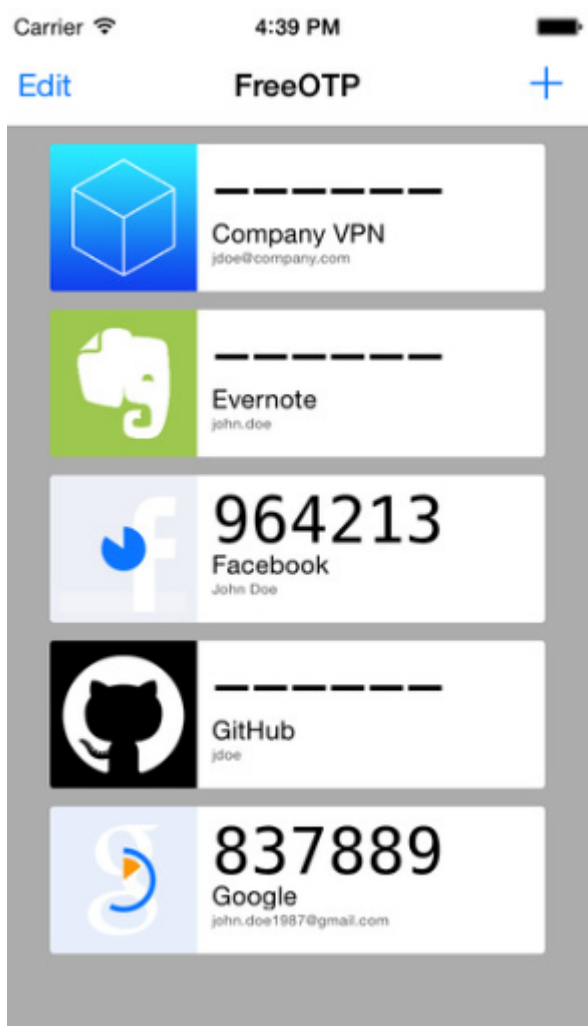
En cas de difficulté ou si votre compte est bloqué contactez votre administrateur 

Installer et configurer la partie client (FreeOTP)

Installez **FreeOTP** sur votre smartphone (pour [Android](#), pour [iOS](#)) et lancez-le.

 **Travail à faire**

Envoyez moi des captures d'écran de vos appareils iPhone et autres applications



Tout en **bas à droite**, appuyez sur le signe '+' pour **ajouter un OTP**.

Il va vous proposer de **Scanner un QR-Code**.



Le QR-Code ci dessus est généré à partir des informations figurant ci-dessous (*que vous auriez pu saisir manuellement*).

- **Issuer:** Lycée Fenelon - BTS SIO SLAM

☒ Paramètre Issuer

STRONGLY RECOMMENDED: The **issuer** parameter is a string value indicating the provider or service this account is associated with, URL-encoded according to [RFC 3986](#).

If the issuer parameter is absent, issuer information may be taken from the issuer prefix of the label. If both issuer parameter and issuer label prefix are present, they should be equal.

- **Account:** BTS SIO SLAM
- **Secret:** JR4WGZLFEBDGK3TFNRXW4IBNEBBFIUZAKNEU6ICTJRAU2

☒ Paramètre Secret

REQUIRED: The **secret** parameter is an arbitrary key value encoded in Base32 according to [RFC 3548](#).

The padding specified in [RFC 3548 section 2.2](#) is not required and should be omitted.

C'est le **secret** qui sera **partagé par le client et le serveur**. Personne ne doit le connaître !

Il doit être aléatoire, et il faut l'entrer en **base32**.

Allez sur le site <https://www.dcode.fr/code-base-32> pour en générer un à votre guise.

Pour le TP, on utilisera la **chaîne** Lycée Fenelon - BTS SIO SLAM

✓ La base 32

La base32 n'est pas très courante, mais vous pouvez y entrer des caractères aléatoires (A-Z 2-7).

Le code Base32 utilise 32 caractères : ABCDEFGHIJKLMNOPQRSTUVWXYZ234567 et le symbole = utilisé optionnellement comme caractère final (comme en [Base64](#)).

La norme indique que le message Base32 doit avoir un nombre de caractères [multiple de 8](#). Le message obtenu doit donc être complété par le caractère = (padding char).

Ce **padding char** n'est **pas exigé** pour le OTP

- **Type:** Time-based (TOTP)

☐ Counter

☒ Timeout

- **Algorithm:** Laissez "SHA256"
- **Interval:** 30. C'est la durée de validité du code OTP en secondes

- **Digits:** Laissez 6.

Travail à faire

Ajoutez ce token à votre application mobile

Vous devez avoir désormais un **rectangle** affichant l'image du service et son nom (*Issuer*).

Cliquez dessus, un code à **6 chiffres** doit apparaître temporairement

Maintenant installons **la partie serveur**.

Côté serveur

Langages

Ce TP se base sur le langage PHP mais la même chose fonctionnerait avec Python, NodeJS...

En PHP

Travail à faire

Sur votre VM distante, dans un dossier de votre **DocumentRoot**, créez un nouveau dossier nommé **PHP-Authentication-OTP**.

Tout le reste du TP se déroulera **dans** ce dossier.

Récupérer et installer la librairie côté serveur

Installez la dépendance **otphp** via **composer**

Dépendance à installer

composer require spomky-labs/otphp

Vérifier et synchroniser l'heure du serveur

Si votre serveur et votre téléphone ne sont **pas synchrones** au niveau de l'heure, les codes TOTP seront valides moins longtemps que prévu, voire pas du tout.

Affichez l'heure de votre serveur, et **vérifiez** que l'heure obtenue soit **exactement** celle de votre téléphone (idéalement à la seconde près).

Dans votre terminal

date

Pour une configuration optimale, réglez le serveur avec des serveurs de temps **ntp**

<https://kairos-data.com/actualites/marche/le-network-time-protocol-comment-ca-marche/>

Dans un fichier `00.checkDateTime.php` ✓

```
<?php
```

```
// Inclusion des dépendances
require_once dirname(__FILE__).'/vendor/autoload.php';
use OTPHP\TOTP;

/*****
 * Affichage du temps pour information
 *****/

// Définition de la zone de temps
date_default_timezone_set('Europe/Paris');
$maintenant = time();

// Affichage de maintenant
echo date('Y-m-d H:i:s',$maintenant);
```

Lancez ce script en mode CLI, et vérifiez l'heure avec celle de votre téléphone.

```
php 00.checkDateTime.php
```



```
niko@BTS-PROF-VM-NC /var/www/html/PHP-Authentification-OTP <master>
$ php 00.checkDateTime.php
2026-03-17 10:27:30
niko@BTS-PROF-VM-NC /var/www/html/PHP-Authentification-OTP <master>
```

Tester la génération d'un TOTP

Maintenant que les heures sont synchronisées, ajoutons la **génération de l'OTP** et affichons le.

Travail à faire

Recopiez le code fourni ci-dessous

Exécutez le en mode **CLI** et en mode **Web**

Vérifiez la correspondance avec le code qui apparaît sur votre application

Ils doivent être **identiques** (si vous êtes synchronisés au niveau du temps, évidemment)

Dans un fichier `01.generateOTP.php` ✓

```
<?php
```

```
// Inclusion des dépendances
require_once dirname(__FILE__).'/vendor/autoload.php';
use OTPHP\TOTP;

/*****
 * Génération d'un secret
 *****/

$otp = TOTP::create();

// Génération à la volée si on ne disposait pas d'un secret
//$secret = $otp->getSecret();

// ou utilisation d'un secret déjà généré par nos soins (pour les tests)
$secret = "JR4WGZLFEBDGK3TFNRXW4IBNEBBFIUZAKNEU6ICTJRAU2";

echo "The OTP secret is: {$secret}\n";

/*****
 * Création du TOTP avec des informations précises
 *****/

$otp = TOTP::create(
    $secret,      // secret utilisé (généré plus haut)
    30,           // période de validité
    'sha256',     // Algorithme utilisé
    6             // 6 digits
);
$otp->setLabel('BTS SIO SLAM'); // The label
$otp->setIssuer('Lycée Fenelon - BTS SIO SLAM');
$otp->setParameter('image', 'https://avatars.githubusercontent.com/u/1199051?v=4'); // FreeOTP
can display image
echo "The current OTP is: {$otp->now()}\n";

/*****
 * Affichage du temps pour information
 *****/

// Définition de la zone de temps
date_default_timezone_set('Europe/Paris');
$maintenant = time();

// Affichage de maintenant
echo date('Y-m-d H:i:s', $maintenant);
```

● Remarquez que nous avons saisi les mêmes paramètres:

- Le secret commun JR4WGZLFEBDGK3TFNRXW4IBNEBBFIUZAKNEU6ICTJRAU2

- L'intervalle à 30 secondes ('interval' ⇒ 30)
- L'algorithme est **SHA256**
- Le code sur 6 chiffres

En mode **CLI**:

```
niko@BTS-PROF-VM-NC /var/www/html/PHP-Authentification-OTP <master>
$ php 01.generateOTP.php
The OTP secret is: JR4WGZLFEBDGK3TFNRXW4IBNEBBFIUZAKNEU6ICTJRAU2
The current OTP is: 262630
2026-03-17 10:38:08%
```

Si l'heure affichée **correspond bien à l'heure de votre smartphone**, la seconde ligne devrait vous afficher un **code identique** à celui que vous affiche l'application **FreeOTP**.

Cette valeur n'est **valide que pendant 30 secondes**. (*attention à la synchronisation du temps*)

Si ce n'est pas le cas

Si ce n'est pas le cas, c'est un manque de bol (*vous avez généré un code juste à la limite de la période de temps*), ou alors vous avez fait une erreur sur l'un de ces trois paramètres:

- Heure exacte entre serveur et smartphone
- Secret commun
- Intervall

En cas de doute, supprimez l'OTP dans l'application **FreeOTP** et re-créez-en un nouveau.

Sécuriser un formulaire de login

Maintenant que le code OTP sont **générés à l'identique** entre serveur et smartphone, nous allons nous en servir pour **ajouter un facteur** à l'authentification sur un formulaire de login existant.

Notre application

Version basique

Nous allons créer une **application fictive** qui n'a qu'un **seul écran** (*l'écran de login*) et capable de faire une seule chose: *Vous dire si votre login et mot de passe sont corrects.*

Travail à faire

Recopiez le code suivant
Exécutez le en mode **Web**
Tentez la connexion

 Dans un fichier `02.unsecureForm.php` ✓

```
<?php
    date_default_timezone_set('Europe/Paris');
?>

<form method="POST">
    Login: <input type="text" name="login"><br>
    Mot de passe: <input type="password" name="password"><br>
    <input type="submit" value="Login"><br>
</form>
<hr>

<?php

/*****
 * Fonction de vérification du formulaire
 *****/

// Fonction qui renvoie true si login et mot de passe sont corrects
function checkLoginPassword($login, $password)
{
    if ($login=='toto' && $password=='titi') return true;
    return false;
}

$formOutput = "";
// Traitement du formulaire de login:
if (!empty($_POST['login']))
{
    if ( checkLoginPassword($_POST['login'], $_POST['password']) )
        $formOutput = "👍 👍 👍 Login OK !";
    else
        $formOutput = "💀 💀 💀 Echec login";
}

echo $formOutput;
```

Vous noterez que notre application est particulièrement simple: Login et mot de passe sont **codés en dur** dans le programme.

Si vous entrez bien *toto/titi*, elle vous répondre: 👍 👍 👍 **Login OK !**

Modification du formulaire de login

Nous allons maintenant ajouter l'OTP.

Ajoutons d'abord un **champ OTP au formulaire** de login.

OTP: `<input type="password" name="otp"><br>`

Type d'input

On pourrait modifier le type d'input HTML.

Ici, on met **password**, mais il est possible de modifier!

Le formulaire devient:

```
<form method="POST">
  Login: <input type="text" name="login"><br>
  Mot de passe: <input type="password" name="password"><br>
  OTP: <input type="password" name="otp"><br>
  <input type="submit" value="Login"><br>
</form>
```

Login:

Mot de passe:

OTP:

Login OK !

Modification du traitement du formulaire

Nous allons ajouter la fonction suivante:

```
// Vérifie la valeur OTP
function checkOTP($otp_form): bool
{
    global $otp;

    return $otp->verify($otp_form);
}
```

Cette fonction renvoie **true** si le code OTP est **valide**.

Ajoutons maintenant la **vérification de l'OTP** à celle du login/mot de passe:

```
if ( checkLoginPassword($_POST['login'], $_POST['password']) && checkOTP( $_POST['otp'] ) )
```

Le code entier devient donc:

 Dans un fichier 03.secureForm.php

```
<?php
```

```
// Inclusion des dépendances
```

```
require_once dirname(__FILE__).'/vendor/autoload.php';
```

```
use OTPHP\TOTP;
```

```
/******
```

```
 * Génération d'un secret
```

```
***** */
```

```
$otp = TOTP::create();
```

```
// $secret = $otp->getSecret();
```

```
// Utilisation d'un secret déjà généré
```

```
$secret = "JR4WGZLFEBDGK3TFNRXW4IBNEBBFIUZAKNEU6ICTJRAU2";
```

```
$secretOutput = "The OTP secret is: {$secret}\n";
```

```
/******
```

```
 * Création du TOTP avec des informations précises
```

```
***** */
```

```
$otp = TOTP::create(
```

```
    $secret,      // secret utilisé (généré plus haut)
```

```
    30,           // période de validité
```

```
    'sha256',     // Algorithme utilisé
```

```
    6             // 6 digits
```

```
);
```

```
$otp->setLabel('BTS SIO SLAM'); // The label
```

```
$otp->setIssuer('Lycée Fenelon - BTS SIO SLAM');
```

```
$otp->setParameter('image', 'https://avatars.githubusercontent.com/u/1199051?v=4'); // FreeOTP  
can display image
```

```
$otpOutput = "The current OTP is: {$otp->now()}\n";
```

```
/******
```

```
 * Affichage du temps pour information
```

```
***** */
```

```
// Définition de la zone de temps
```

```
date_default_timezone_set('Europe/Paris');
```

```
$maintenant = time();
```

```
// Affichage de maintenant
```

```
$dateOutput = date('Y-m-d H:i:s', $maintenant);
```

```
/******
```

```
 * Génération du QRCode
```

```
***** */
```

```
// Note: You must set label before generating the QR code
```

```

$qrCodeUri = $otp->getQrCodeUri(
    'https://api.qrserver.com/v1/create-qr-code/?data=[DATA]&size=300x300&ecc=M',
    '[DATA]'
);
$qrCodeOutput = "<img src='{ $qrCodeUri }'>";

/*****
* Fonction de vérification du formulaire
*****/

// Fonction qui renvoie true si login et mot de passe sont corrects
function checkLoginPassword($login, $password)
{
    if ($login=='toto' && $password=='titi') return true;
    return false;
}

// Vérifie la valeur OTP
function checkOTP($otp_form): bool
{
    global $otp;

    return $otp->verify($otp_form);
}

$formOutput = "";
// Traitement du formulaire de login:
if (!empty($_POST['login']))
{
    if ( checkLoginPassword($_POST['login'], $_POST['password']) && checkOTP( $_POST['otp'] ) )
        $formOutput = " 🍷 🍷 🍷 Login OK !";
    else
        $formOutput = " 💀 💀 💀 Echec login";
}
?>

<!DOCTYPE HTML>
<html>
    <head>
        <title>OTP QR CODE</title>
    </head>
    <body>
        <h1>QR Code</h1>
        <div>
            <span>Secret: <?= $secretOutput; ?></span>

        <div>
            <span>Current OTP: <?= $otpOutput; ?></span>
        </div>
    </body>
</html>

```

```

<div>
  <span>Date: <?= $dateOutput; ?></span>
</div>

<div>
  <?= $qrCodeOutput; ?>
</div>

<form method="POST">
  Login: <input type="text" name="login"><br>
  Mot de passe: <input type="password" name="password"><br>
  OTP: <input type="password" name="otp"><br>
  <input type="submit" value="Login"><br>
</form>
<hr>
<h2>Retour du formulaire <br><br> <b><?= $formOutput; ?></b></h2>

</body>
</html>

```

Travail à faire

Recopiez le code ci-dessus

Exécutez le en mode **Web**

Testez le **vrai**

Le bon login, le bon mot de passe, et le code OTP fourni par **FreeOTP**

Votre login devrait être accepté

Testez le **faux**

Vous pouvez refaire le test avec le **bon login**, le **bon mot de passe** mais un code OTP 6 chiffres au hasard.

Votre login sera refusé même si le mot de passe est bon

Conclusion

C'est tout l'intérêt d'OTP!

- Si une personne vous vole votre mot de passe, elle ne pourra pas l'utiliser pour se connecter car elle n'a pas le générateur OTP de votre smartphone.
- Elle n'a pu voler qu'un **seul des deux facteurs** (*ce que je sais*).
- Cela est utile si le mot de passe est volé sur le réseau, ou sur la machine (*keylogger*) ou si quelqu'un vous voit le taper.

Conseils

- Le code OTP ne doit **jamais remplacer le mot de passe**. Il ne peut que le **compléter**
- Bien entendu, on ne laisse **jamais** les mots de passe et secrets OTP **directement dans les sources**. Stockez les ailleurs !
- Je vous **recommande** de toujours **afficher la date de l'heure** sur votre formulaire de login.
 - Vous pourrez ainsi immédiatement voir s'il y a un **décalage** avec votre téléphone.
- Il est facile de mettre la vérification OTP sous forme d'une librairie bien à part que vous pourrez appeler depuis différentes applications php. La double authentification sera alors facile à ajouter à la plupart des applications php existantes:
 - Simple ajout d'un champ OTP dans le formulaire de login.
 - Un **require_once** pour inclure votre librairie.
 - Appel à votre fonction checkOTP() là où le logiciel vérifie login et mot de passe.
- Possibilité de générer un OTP spécifique **par utilisateur**. Ainsi, le champ OTP apparaît après la première étape du mot de passe.
 - Le OTP aurait un **secret basé sur le login** de l'utilisateur.

Auto évaluation

Pourquoi?

Cette section est là pour vous auto évaluer sur cette séance.

1. Compréhension des concepts de base

- Pourquoi l'authentification à deux facteurs (2FA) est-elle considérée comme plus sécurisée que l'authentification par mot de passe seul ?
- Quels sont les trois facteurs d'authentification classiques ? Donnez un exemple concret pour chacun.
- Pourquoi le protocole TOTP est-il préféré à HOTP dans ce TP ? Quels sont les avantages et inconvénients de chaque méthode ?

2. Mise en œuvre technique

- Pourquoi est-il crucial que le serveur et le smartphone soient synchronisés en temps pour que TOTP fonctionne correctement ?
- Que se passe-t-il si le secret partagé entre le serveur et l'application mobile est compromis ? Comment pourrait-on atténuer ce risque ?
- Pourquoi le secret OTP doit-il être encodé en base32 ? Que se passerait-il si on utilisait un encodage différent (comme base64) ?

3. Sécurité et bonnes pratiques

- Pourquoi est-il déconseillé de stocker les mots de passe et les secrets OTP directement dans le code source ? Où et comment devraient-ils être stockés ?
 - Comment pourriez-vous améliorer la sécurité de l'application si un utilisateur perd son téléphone (et donc son application TOTP) ?
 - Quels sont les risques si un attaquant parvient à voler à la fois le mot de passe et le code OTP d'un utilisateur ? Comment pourrait-on se prémunir contre ce scénario ?
-

4. Application pratique

- Si vous deviez implémenter ce système pour un site web réel, quelles étapes supplémentaires ajouteriez vous pour sécuriser davantage le processus (par exemple, sauvegarde des codes de récupération, notifications en cas de tentative de connexion suspecte, etc.) ?
 - Comment adapteriez vous ce TP pour que chaque utilisateur ait son propre secret OTP, lié à son compte ?
 - Comment pourriez-vous intégrer ce système dans une application existante (par exemple, un site de réservation de billets pour un festival) sans perturber l'expérience utilisateur ?
-

5. Réflexion critique

- Quels sont les points faibles de l'authentification par OTP via application mobile ? Comment pourraient-ils être exploités par un attaquant ?
 - Dans quels cas l'authentification biométrique pourrait-elle être préférable à l'OTP ? Et inversement ?
 - Comment expliqueriez vous à un utilisateur non technique l'importance de ne pas partager son code OTP, même avec une personne de confiance ?
-

6. Extension du projet

- Comment pourriez-vous générer automatiquement un QR code pour chaque utilisateur, afin qu'il puisse configurer son application TOTP sans avoir à entrer manuellement le secret ?
 - Comment pourriez-vous permettre à un utilisateur de désactiver ou réinitialiser son OTP en cas de perte de son téléphone ?
 - Comment testeriez vous la robustesse de votre implémentation OTP (par exemple, en simulant des attaques par force brute ou des désynchronisations horaires) ?
-

7. Perspectives d'amélioration

- Comment pourriez-vous rendre ce système plus convivial pour les utilisateurs (par exemple, en permettant l'utilisation de plusieurs appareils pour générer des codes OTP) ?

- Quels autres mécanismes de sécurité pourriez-vous combiner avec l'OTP pour renforcer encore la protection des comptes (par exemple, listes blanches d'adresses IP, vérification par email, etc.) ?
-

8. Cas concrets

- Imaginez que vous gérez un site sensible (comme une banque en ligne). Quels mécanismes supplémentaires mettriez-vous en place pour sécuriser l'authentification, en plus de l'OTP ?
- Si un utilisateur vous contactait pour dire qu'il ne parvient pas à se connecter car son code OTP ne fonctionne pas, quelles étapes suivriez-vous pour diagnostiquer et résoudre le problème ?